



Python Programming Fundamentals

pytest Basics

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. Installing pytest
2. Naming Conventions
3. The assert Statement
4. Writing Your First Tests
5. Running pytest
6. Reading pytest Output
7. Testing Validation
8. Testing the BankAccount



Outline

- 1. Installing pytest**
2. Naming Conventions
3. The assert Statement
4. Writing Your First Tests
5. Running pytest
6. Reading pytest Output
7. Testing Validation
8. Testing the BankAccount



1. Installing pytest

```
pip install pytest
```

Or inside a virtual environment:

```
python -m pip install pytest
```

```
# Verify installation
```

```
pytest --version
```

In VS Code: open the Command Palette, search “Python: Configure Tests”, select pytest, choose your test folder.



Outline

1. Installing pytest
- 2. Naming Conventions**
3. The assert Statement
4. Writing Your First Tests
5. Running pytest
6. Reading pytest Output
7. Testing Validation
8. Testing the BankAccount



2. Naming Conventions

pytest uses **discovery** — it automatically finds tests if you follow the conventions:

Convention	Example
Test files start with test_	test_product.py
Test functions start with test_	test_valid_price()
Test classes start with Test	TestProduct

Typical folder structure:

```
lab13/
├── product.py
├── bank_account.py
└── tests/
    ├── test_product.py
    └── test_bank_account.py
```



Outline

1. Installing pytest
2. Naming Conventions
- 3. The `assert` Statement**
4. Writing Your First Tests
5. Running pytest
6. Reading pytest Output
7. Testing Validation
8. Testing the BankAccount



3. The assert Statement

```
# assert checks that a condition is True
# If False, the test FAILS with an AssertionError

assert 1 + 1 == 2          # passes
assert "hello".upper() == "HELLO" # passes

assert [1, 2, 3][1] == 2    # passes
assert isinstance(42, int)  # passes

# Failing assert – with optional message
value = compute_something()
assert value > 0, f"Expected positive, got {value}"
```




Outline

1. Installing pytest
2. Naming Conventions
3. The assert Statement
- 4. Writing Your First Tests**
5. Running pytest
6. Reading pytest Output
7. Testing Validation
8. Testing the BankAccount



4. Writing Your First Tests

```
# tests/test_product.py
from product import Product

def test_product_name():
    p = Product("Apple", 1.99, 101)
    assert p.get_name() == "Apple"

def test_product_price():
    p = Product("Apple", 1.99, 101)
    assert p.get_price() == 1.99

def test_product_id():
    p = Product("Apple", 1.99, 101)
    assert p.get_product_id() == 101

def test_product_str():
```



4. Writing Your First Tests

```
p = Product("Apple", 1.99, 101)
assert str(p) == "Product [101]: Apple – €1.99"
```



Outline

1. Installing pytest
2. Naming Conventions
3. The assert Statement
4. Writing Your First Tests
- 5. Running pytest**
6. Reading pytest Output
7. Testing Validation
8. Testing the BankAccount



5. Running pytest

From the terminal (in your project folder):

```
# Run all tests  
pytest
```

```
# Run a specific file  
pytest tests/test_product.py
```

```
# Verbose output (shows each test name)  
pytest -v
```

```
# Short traceback  
pytest --tb=short
```



Outline

1. Installing pytest
2. Naming Conventions
3. The assert Statement
4. Writing Your First Tests
5. Running pytest
- 6. Reading pytest Output**
7. Testing Validation
8. Testing the BankAccount



6. Reading pytest Output

```
===== test session starts =====
```

```
collected 4 items
```

```
tests/test_product.py::test_product_name PASSED [25%]
```

```
tests/test_product.py::test_product_price PASSED [50%]
```

```
tests/test_product.py::test_product_id PASSED [75%]
```

```
tests/test_product.py::test_product_str FAILED [100%]
```

```
===== FAILURES =====
```

```
test_product_str - AssertionError:
```

```
    assert 'Product [101]: Apple – €1.99' ==
```

```
        'Product [101]: Apple - €1.99'
```

```
    ← em dash vs hyphen!
```



Outline

1. Installing pytest
2. Naming Conventions
3. The assert Statement
4. Writing Your First Tests
5. Running pytest
6. Reading pytest Output
- 7. Testing Validation**
8. Testing the BankAccount



7. Testing Validation

```
import pytest
from product import Product

def test_invalid_price_raises_value_error():
    with pytest.raises(ValueError):
        Product("Apple", -1.0, 101)

def test_zero_price_raises_value_error():
    with pytest.raises(ValueError):
        Product("Apple", 0.0, 101)

def test_empty_name_raises_value_error():
    with pytest.raises(ValueError):
        Product("", 1.99, 101)

def test_whitespace_name_raises_value_error():
```



7. Testing Validation

```
with pytest.raises(ValueError):  
    Product("    ", 1.99, 101)
```



Outline

1. Installing pytest
2. Naming Conventions
3. The assert Statement
4. Writing Your First Tests
5. Running pytest
6. Reading pytest Output
7. Testing Validation
- 8. Testing the BankAccount**



8. Testing the BankAccount

```
from bank_account import BankAccount

def test_initial_balance():
    account = BankAccount("Alice", 100.0)
    assert account.get_balance() == 100.0

def test_deposit():
    account = BankAccount("Alice", 100.0)
    account.deposit(50.0)
    assert account.get_balance() == 150.0

def test_withdraw():
    account = BankAccount("Alice", 100.0)
    account.withdraw(30.0)
    assert account.get_balance() == 70.0
```



8. Testing the BankAccount

```
def test_overdraft_raises():  
    account = BankAccount("Alice", 50.0)  
    with pytest.raises(ValueError):  
        account.withdraw(100.0)
```



Thanks for Watching - Any questions?

